

LG-CISH: Encoding and Decoding Algorithms

Lossless Coverless Image Steganography via CLIP Semantic Hashing,
Base- N Positional and Lehmer-Permutation Coding

Ayushman Bhattacharya Soumyashree Biswas Piyali Sarkar Harsh Raj

1 Notation and Shared State

Sender and receiver share, offline, an image database, the CLIP image encoder Φ , and the resulting *codebook* $\mathcal{C} = \{c_0, \dots, c_{N-1}\}$ of N mutually distinct images ($N=40$ in our deployment). Let $\Phi : \mathcal{X} \rightarrow \mathbb{R}^d$ ($d=512$). Each image x is represented by its ℓ_2 -normalised embedding

$$e(x) = \frac{\Phi(x)}{\|\Phi(x)\|_2}, \quad \|e(x)\|_2 = 1, \quad (1)$$

and the similarity of two normalised embeddings is the cosine (inner) product

$$s(a, b) = \langle a, b \rangle = a^\top b \in [-1, 1]. \quad (2)$$

The codebook stores the matrix $\mathbf{C} \in \mathbb{R}^{N \times d}$ of row embeddings $\mathbf{C}_j = e(c_j)$. It is built so that all images are well separated, $\max_{i \neq j} s(\mathbf{C}_i, \mathbf{C}_j) \leq \tau$ (we use $\tau=0.85$); Algorithm 1 performs greedy pruning to enforce this.

2 Cryptographic Framing

Before coding, the message M is compressed, integrity-tagged, and optionally encrypted. With ZLIB the lossless compressor, CRC32 the 4-byte CRC, and AES_κ AES-256-CBC under key κ (a random 16-byte IV is prepended to the ciphertext), the framed payload is

$$p_0 = \text{ZLIB}(\text{UTF8}(M)), \quad f = \text{AES}_\kappa(p_0 \parallel \text{CRC32}(p_0)), \quad (3)$$

where the AES step is identity when no key is supplied (CRC only). The inverse, UNWRAP_κ , decrypts, splits off the trailing CRC, and *rejects* the message if the recomputed CRC disagrees (probability of an undetected error $\approx 2^{-32}$).

3 Channel 1: Base- N Positional Coding

The codebook is a radix- N alphabet, so the information-theoretic capacity is

$$C_{\text{base}} = \log_2 N \text{ bits/image} \quad (N=40 \Rightarrow 5.322 \text{ bits/image}). \quad (4)$$

The framed bytes f are mapped to a single non-negative integer with a `0x01` sentinel byte prepended to preserve leading zeros,

$$v = \text{INT}_{\text{BE}}(0x01 \parallel f), \quad (5)$$

Algorithm 1 Codebook construction

1: **Input:** candidate image set $\mathcal{D}$, encoder Φ , threshold τ
2: **Output:** codebook $\mathbf{C}$, image paths, size N
3: $E \leftarrow [e(x) \text{ for } x \in \mathcal{D}]$ $\triangleright$ Eq. (1)
4: $\mathcal{K} \leftarrow \emptyset$
5: **for** $j = 0$ **to** $|\mathcal{D}| - 1$ **do**
6: **if** $\max_{i \in \mathcal{K}} s(E_i, E_j) \leq \tau$ **then** $\mathcal{K} \leftarrow \mathcal{K} \cup \{j\}$
7: **end for**
8: **return** $\mathbf{C} \leftarrow E[\mathcal{K}]$, $\text{paths}[\mathcal{K}]$, $N \leftarrow |\mathcal{K}|$

Algorithm 2 Base- N integer coding: TOBASEN / FROMBASEN

1: **function** TOBASEN(v, N)
2: $D \leftarrow []$; **while** $v > 0$: $D.\text{append}(v \bmod N)$; $v \leftarrow \lfloor v/N \rfloor$
3: **return** REVERSE(D) $\triangleright$ MSB first; Eq. (6)
4: **function** FROMBASEN(D, N)
5: $v \leftarrow 0$; **for** d **in** D : $v \leftarrow vN + d$
6: **return** v

then written in radix N (most-significant digit first, always non-zero):

$$v = \sum_{k=0}^{L-1} d_k N^k, \quad d_k \in \{0, \dots, N-1\}, \quad L = \lfloor \log_N v \rfloor + 1. \quad (6)$$

Digit d_k selects codebook image c_{d_k} . The map $f \leftrightarrow (d_0, \dots, d_{L-1})$ is bijective, hence *lossless*. Algorithm 2 gives the integer $\leftrightarrow$ digit conversion.

4 Channel 2: Lehmer-Permutation Coding

Base- N coding reaches the $L \log_2 N$ ceiling but lets digits (images) *repeat*, which can make a cover look unnatural. Permutation coding instead emits, per block of B images, B *distinct* codebook images selected without repetition. A block carries

$$C_{\text{perm}} = \frac{1}{B} \log_2 P(N, B), \quad P(N, B) = \frac{N!}{(N-B)!} = \prod_{t=0}^{B-1} (N-t), \quad (7)$$

bits/image (e.g. $N=40, B=8 \Rightarrow 5.187$; $B=N \Rightarrow 3.979$). Within a block, the t -th image is identified by its *rank* r_t among the still available images (mixed-radix / Lehmer code with radix $N-t$):

$$u = \sum_{t=0}^{B-1} r_t \prod_{j=t+1}^{B-1} (N-j), \quad 0 \leq r_t < N-t. \quad (8)$$

The whole integer v (Eq. (5)) is first written in radix $m = P(N, B)$, giving block values $(e_0, \dots, e_{L'-1})$. Each block value is *whitened* by a deterministic shared keystream to avoid a tell-tale sorted prefix from the Lehmer leading-zero artifact:

$$\tilde{e}_i = (e_i + \mu_i) \bmod m, \quad \mu_i = \text{SHA256}(\text{"LG-CISH-perm-whiten-"}i) \bmod m. \quad (9)$$

Whitening is invertible mod m and within-block distinctness is preserved, so the channel stays lossless. Algorithm 3 encodes/decodes one block.

Algorithm 3 Lehmer block coding: PERMEncode / PERMDecode

```
1: function PERMEncode( $u, N, B$ )  $\triangleright$  integer  $\rightarrow B$  distinct indices
2:   for  $t = B - 1$  down to 0:  $r_t \leftarrow u \bmod (N - t)$ ;  $u \leftarrow \lfloor u / (N - t) \rfloor$ 
3:    $A \leftarrow [0, 1, \dots, N - 1]$ ; return  $[A.\text{pop}(r_t) \text{ for } t = 0 \dots B - 1]$ 
4: function PERMDecode( $\langle a_0, \dots, a_{B-1} \rangle, N$ )  $\triangleright$  indices  $\rightarrow$  integer
5:    $A \leftarrow [0, \dots, N - 1]$ ;  $u \leftarrow 0$ 
6:   for  $t = 0$  to  $B - 1$ :  $r \leftarrow A.\text{index}(a_t)$ ;  $u \leftarrow u(N - t) + r$ ;  $A.\text{pop}(r)$ 
7:   return  $u$   $\triangleright$  Eq. (8)
```

Algorithm 4 ENCODE: message $\rightarrow$ ordered image sequence

```
1: Input: message  $M$ ; codebook  $\mathcal{C}$  (size  $N$ ); key  $\kappa$ ; flags compress, perm, block  $B$ , alias
2:  $p_0 \leftarrow \text{ZLIB}(\text{UTF8}(M))$  if compress else  $\text{UTF8}(M)$ 
3:  $f \leftarrow \text{AES}_\kappa(p_0 \parallel \text{CRC32}(p_0))$   $\triangleright$  Eq. (3); AES optional
4:  $v \leftarrow \text{INT}_{\text{BE}}(0x01 \parallel f)$   $\triangleright$  Eq. (5)
5: if perm then
6:    $m \leftarrow P(N, B)$ ;  $(e_0, \dots, e_{L'-1}) \leftarrow \text{TOBASEN}(v, m)$ 
7:    $\text{idx} \leftarrow \text{CONCAT}_i \text{PERMEncode}((e_i + \mu_i) \bmod m, N, B)$   $\triangleright$  Eq. (9)
8: else  $\text{idx} \leftarrow \text{TOBASEN}(v, N)$   $\triangleright$  Eq. (6)
9: if alias then  $S \leftarrow [\text{CHOOSEALIAS}(j) \text{ for } j \in \text{idx}]$  else  $S \leftarrow [c_j \text{ for } j \in \text{idx}]$ 
10: return image sequence  $S$ , metadata
```

5 Optional: Plausibility Aliases

Each codebook slot j may own several visually interchangeable candidate images (LLM-verified aliases) that all map to the same index j . When enabled, the encoder picks, per emitted index, the candidate that makes the overall sequence look most like a natural album; because every alias of slot j decodes to j , the decoder is unaffected and losslessness is preserved.

6 Top-Level Encode and Decode

Algorithm 4 is the sender pipeline (Eqs. (3)–(9)); Algorithm 5 is its exact inverse. Decoding recovers each received image’s index by CLIP nearest neighbour,

$$\hat{i}_t = \arg \max_{0 \leq j < N} s(e(\tilde{x}_t), \mathbf{C}_j), \quad (10)$$

where $\tilde{x}_t$ is the (possibly channel-distorted) t -th received image. The reliability of (10) is the per-image *decoding margin*

$$\gamma_t = s_{(1)} - s_{(2)}, \quad (11)$$

the gap between the top-1 and top-2 cosine similarities; $\gamma_t > 0$ guarantees the correct image wins and hence BER = 0.

7 Losslessness

Every transformation above is a bijection: (3) is invertible (compression and AES are lossless, CRC is appended), (5) is invertible by the sentinel, (6)/(7) are positional/factorial number systems, and

Algorithm 5 DECODE: image sequence $\rightarrow$ message

```
1: Input: received images  $\langle \tilde{x}_0, \dots, \tilde{x}_{T-1} \rangle$ ; codebook  $\mathbf{C}$ ; key  $\kappa$ ; flags compress, perm,  $B$ 
2: for  $t = 0$  to  $T - 1$ :  $\hat{i}_t \leftarrow \arg \max_j s(e(\tilde{x}_t), \mathbf{C}_j)$   $\triangleright$  Eq. (10)
3: if perm then
4:    $m \leftarrow P(N, B)$ ; for each block  $i$ :  $e_i \leftarrow (\text{PERMDECODE}(\hat{i}_{[iB:iB+B]}, N) - \mu_i) \bmod m$ 
5:    $v \leftarrow \text{FROMBASEN}((e_0, \dots), m)$ 
6: else  $v \leftarrow \text{FROMBASEN}(\hat{i}, N)$ 
7:  $f \leftarrow \text{STRIPSENTINEL}(\text{BYTES}_{\text{BE}}(v))$ 
8:  $p_0 \leftarrow \text{UNWRAP}_{\kappa}(f)$   $\triangleright \text{AES}^{-1}$  then verify CRC-32; reject on mismatch
9: return  $\text{UTF8}^{-1}(\text{UNZIP}(p_0))$  if compress else  $\text{UTF8}^{-1}(p_0)$ 
```

(9) is invertible mod m . Therefore, whenever CLIP recovers every index correctly ($\gamma_t > 0 \forall t$, Eq. (11)), $\text{DECODE}(\text{ENCODE}(M)) = M$ exactly, i.e. $\text{BER} = 0$.